InterSystems
Creative data technology

# Using Large Language Models (LLMs) with FHIR – Concepts, Exercises & Safety Foundations

Patrick W. Jamieson, M.D.
Technical Product Manager

Russ Leftwich, M.D.
Senior Clinical Advisor, Interoperability

# The AI+FHIR Series

This is the first lesson in a four-part series that takes you from AI foundations to production-ready healthcare AI applications.

| | | |
|---|---|---|
| **This Lesson** | AI foundations, hands-on exercises, safety framing | *Patient summarization CLI pipeline* |
| **Agentic AI Part One** | Agent architecture, MCP, tool-use, structured validation | *FHIR-connected AI agent with audit logging* |
| **Agentic AI Part Two** | FHIR as knowledge layer, subagents, human review | *Multi-agent clinical workflow* |
| **RAG Lab** | Embeddings, vector stores, retrieval-augmented generation | *Retrieval pipeline grounded in patient data* |

# Learning Objectives

1. Explain core LLM concepts (tokens, embeddings, context windows, temperature, prompts, hallucination) using healthcare analogies
2. Set up and interact with a local LLM (Ollama) from Python
3. Transform a FHIR patient bundle into LLM-ready input through resource grouping and feature extraction
4. Build a patient summarization pipeline that reduces ~100K tokens to ~1K tokens
5. Apply prompt engineering techniques and observe how they change LLM output
6. Articulate AI safety and governance principles for healthcare AI

# Lesson Structure: Three Phases

## Phase 1: AI Foundations

9 core concepts with healthcare analogies
What is an LLM? Tokens, context windows, embeddings, temperature, prompts, hallucination, local vs. cloud, model selection

## Phase 2: Hands-On Exercises

Exercise 1: LLM Hello World
Exercise 2: FHIR Bundle Exploration
Exercise 3: Patient Summarization Pipeline
Exercise 4: Prompt Engineering Comparison

## Phase 3: Safety & Series Bridge

The Golden Rule of validation
Hallucination risks, PHI/HIPAA, human-in-the-loop
Regulatory awareness (ONC HTI-1, FDA)
How Lesson connects to the rest of the series

PHASE 1

# AI Foundations

9 core concepts, grounded in a healthcare
No prior AI/ML knowledge required

# 1. What Is a Large Language Model?

A software system trained on vast amounts of text to predict what comes next in a sequence. Given a prompt, it generates a response word by word, selecting each word based on patterns learned during training.

**Healthcare Analogy**

Think of an LLM as a clinician who has read every textbook, every journal article, and every clinical note ever written — but has never seen an actual patient. The knowledge is vast but indirect: learned from text, not from lived clinical experience.

LLMs process text patterns — they don't "understand" FHIR resources like a FHIR server does

An LLM has no database, no schema validation — only patterns in language

When we combine LLMs with FHIR, we provide the structured data the LLM lacks

# 2. Tokens and Tokenization

Before an LLM can process text, it breaks the input into tokens — the fundamental units the model reads. A token is typically a word or subword fragment.

```
"The patient's hemoglobin A1c was 7.2%"
   ["The", " patient", "'s", " hem", "oglobin", " A", "1", "c", " was", " 7", ".", "2", "%"]
(13 tokens)
```

### Pharmacist Reading a Prescription

A pharmacist parses a prescription into discrete units: drug name, dosage, route, frequency. Token boundaries don't always match word boundaries.

### Why This Matters for FHIR:

A Patient/$everything bundle = tens of thousands of tokens

Raw FHIR JSON is token-inefficient (braces, quotes, URLs)

Token count determines if data fits in the LLM's window

Exercise 2 demonstrates this with real numbers

# 3. Context Windows

The context window is the maximum number of tokens an LLM can process in a single interaction — both input and output must fit.

| | |
|---|---|
| **Small local (3B)** | 4K–8K tokens |
| **Medium local (7B)** | 8K–32K tokens |
| **Large cloud (GPT-4, Claude)** | 128K–1M+ tokens |

### Clinician Working Memory

A physician reviewing a complex chart can't hold the entire record in mind at once. They focus on a subset: recent visits, active problems, current medications. The context window is the LLM's equivalent of this limit.

### The Problem (Exercise 2 reveals this):

A typical Synthea patient bundle:
- ~310,000 characters
- ~101,000 tokens
- ~1,263% of an 8K window

The data is 12x too large for a small model's context window.

This is why Exercise 3 extracts features — reducing 101K tokens to ~1K tokens (99% reduction).

# 4. Embeddings — Meaning as Numbers

An embedding is a list of numbers (a vector) that represents the meaning of a word, sentence, or document. Words with similar meanings have similar vectors.

"hypertension" ~ "high blood pressure" (close in embedding space)
"hypertension" ≠ "appendectomy" (far apart in embedding space)

Like how clinicians know "MI" = "myocardial infarction" = "heart attack"
— embeddings give computers this same ability.

*We introduce the concept here; the RAG Lab covers embeddings, vector stores, and similarity search in depth. Embeddings are the foundation of Retrieval-Augmented Generation (RAG) — finding relevant documents by comparing vector similarity before sending them to an LLM.*

# 5. Temperature and Sampling

Temperature controls how the LLM chooses its next word. Low = focused and predictable. High = creative and variable.

**Low (0.1)**

*Confident Specialist*

Almost always picks the most likely word. Deterministic, focused. Like a specialist following established guidelines.

**Medium (0.7)**

*Experienced Clinician*

Balanced — mostly follows evidence but exercises clinical judgment. Default for most tasks.

**High (1.0+)**

*Creative Brainstormer*

Samples more broadly. Creative, diverse, sometimes incoherent. Like generating a differential diagnosis — casts a wide net.

**For clinical summarization, lower temperature (0.1–0.3) is typically safer. Exercise 4 demonstrates this side-by-side.**

# 6. Prompts and System Roles

## System Prompt (Standing Orders)

Sets the LLM's role and behavior rules

Persists across the conversation

"You are a clinical AI assistant"

"Do not invent information"

## User Prompt (Clinical Question)

The specific question or data for this interaction

"Summarize this patient's conditions"

Changes with each request

Contains the actual FHIR data

### Healthcare Analogy — Clinical Protocols

A system prompt is like a hospital policy: "Always verify allergies before prescribing." It shapes every interaction without being restated. The user prompt is the specific clinical question: "What are this patient's active conditions?"

Prompt Engineering Techniques: 1) Role specification 2) Output format 3) Safety constraints 4) Few-shot examples
Exercise 4 compares these side-by-side.

# 7. Hallucination

Hallucination: when an LLM generates plausible-sounding but factually incorrect or entirely fabricated information — with full confidence.

### Healthcare Analogy — Clinical Confabulation

In neurology, confabulation is when a patient with memory impairment fills gaps with fabricated memories, presented with full confidence. LLM hallucination works the same way.

### Danger in Healthcare AI:

Invents a medication the patient never took

Fabricates a lab value not in the FHIR record

Conflates two patients' data in batch processing

Cites a clinical guideline that doesn't exist

### Mitigation Strategies:

Grounding: provide actual FHIR data

"Do not invent" system prompt instructions

Human-in-the-loop validation

RAG: retrieve source docs first (RAG Lab)

# 8. Local vs. Cloud Models

## Local Models (Ollama, LM Studio)

Data never leaves your machine

No BAA needed — PHI stays local

Limited by your hardware (3B–13B typical)

Free after hardware investment

Always available (your machine)

## Cloud Models (GPT-4, Claude, Gemini)

Data sent to third-party servers

Requires BAA for PHI

Access to largest models (100B+ params)

Pay per token

Depends on service uptime

### Healthcare Analogy — Local Lab vs. Reference Lab

Local lab: fast turnaround, specimens stay on-site (privacy), but limited test menu. Reference lab: specialized tests, but specimens shipped off-site, slower, more expensive.
Our exercises use Ollama (local) — even with synthetic data, we practice the privacy-first pattern.

# 9. Model Selection Basics

| | | | |
|---|---|---|---|
| **Small (1B–3B)** | *Llama 3.2 1B/3B, Phi-3 Mini* | Fast, runs on laptops, 8GB RAM | Limited reasoning, shorter outputs |
| **Medium (7B–13B)** | *Llama 3.1 8B, Mistral 7B* | Good balance of quality and speed | May struggle with complex reasoning |
| **Large (30B–70B)** | *Llama 3.1 70B, Mixtral 8x7B* | Strong reasoning, nuanced | Requires 32–64GB+ RAM or GPU |
| **Cloud-scale (100B+)** | *GPT-4, Claude, Gemini* | State-of-the-art capability | Cloud-only, cost, privacy concerns |

**For this course: We use llama3.2:3b via Ollama — small enough for any modern laptop, capable enough for summarization and prompt engineering. Change LLM_MODEL in .env to try larger models.**

PHASE 2

# Hands-On Exercises

4 exercises that progressively build from LLM connectivity
to a complete FHIR patient summarization pipeline

# Exercise 1: LLM Hello World

*Verify your local LLM setup and see your first AI response from Python.*

```
$ python exercise1_llm_chat.py

=== Exercise 1: LLM Hello World ===

Sending prompt to llama3.2:3b...

Response:
An A1c test, also called a hemoglobin
A1c or HbA1c test, measures your
average blood sugar level over the
past 2-3 months...
```

Setup: Ensure Ollama is running (ollama serve) with the model loaded.
Run: python exercise1_llm_chat.py

## What to Notice:

System prompt sets the role:
"clinical AI assistant"

User prompt asks a specific question:
"Explain what an A1c test measures"

The LLM generates natural language from a structured prompt

If you get a connection error, Ollama isn't running — start it first

# Exercise 2: FHIR Bundle Exploration

*How large is a real FHIR patient bundle? Can it fit in a context window?*

```
$ python exercise2_fhir_explore.py

=== FHIR Bundle Statistics ===
Total characters:     310,482
Estimated tokens:     ~101,049
Total resources:      224

=== Resource Breakdown ===
ExplanationOfBenefit    72
Encounter               47
Observation             43
Claim                   25
Condition               14
...

=== Context Window Analysis ===
This bundle is ~1,263% of an
8,000-token context window.
```

**The Key Insight:**

One patient's data is 12x too large for a small model's context window.

We can't just paste FHIR JSON into an LLM.
We need a smarter approach...

**Discussion: What Are Our Options?**

1. Bigger model (expensive, cloud, privacy)
2. Extract & reduce (Exercise 3)
3. Retrieve relevant parts (RAG Lab)

# Exercise 3: Patient Summarization Pipeline

*The core exercise — transform a FHIR bundle into a clinical narrative using a local LLM.*

**1. FETCH**

Retrieve Patient/$everything bundle from FHIR server

**2. GROUP**

Organize resources by type: Condition, Observation, MedicationRequest, etc.

**3. EXTRACT**

Pull clinically relevant features from each type: demographics, conditions, labs, medications

**4. SUMMARIZE**

Send extracted features (~1K tokens) to LLM with clinical system prompt

**5. OUTPUT**

Print clinical narrative summary and token reduction statistics

**Result: ~101,000 tokens --> ~1,040 tokens (99% reduction)**
**The LLM receives only clinically relevant features, not raw JSON. Run: python exercise3_summarizer.py**

# Exercise 3: The "Do Not Invent" Instruction

```
System Prompt (from exercise3_summarizer.py):

"You are a clinical AI assistant reviewing FHIR patient data.
 Provide a concise clinical summary organized by:
 Demographics, Active Conditions, Current Medications,
 Recent Observations, Recent Encounters, Allergies, and
 Notable Findings.

 Do not invent or assume information not present in the data."
```

### Why Is This Instruction Critical?

Without it, the LLM may fabricate data

Example: invents Metformin for a diabetic patient who was never prescribed it

Clinician acting on hallucinated data → medication reconciliation error

This is a prompt-level safety guardrail

### Discussion Question:

"Could a clinician use this summary directly in patient care?"

Answer: No — it requires human verification against source data.

This is the "golden rule" we'll discuss in Phase 3.

# Exercise 4: Prompt Engineering Comparison

*Same patient data, different prompts — see how prompt design changes everything.*

**Strategy 1: Minimal Prompt**

*"You are a helpful assistant."*
*"Summarize this patient data."*

Unstructured paragraph, no consistent format, may hallucinate

**Strategy 2: Structured Prompt**

*Clinical role + section format +*
*"do not invent" constraint*

Organized sections matching requested format, safer, more predictable

**Strategy 3: Few-Shot**

*Includes an example input/output pair before the real data*

Mimics the example format and style without explicit format instructions

Also includes temperature comparison (0.1, 0.7, 1.0) on Strategy 2. Prompt constants are at the top of the file — students can edit and re-run.

Run: python exercise4_prompt_compare.py

# Exercise 4: Temperature Comparison

*Same prompt (Strategy 2), same data — only temperature changes.*

**Temperature 0.1**

*Focused, deterministic*

Nearly identical output each run. Picks the highest-probability words consistently. Best for clinical summarization where accuracy and reproducibility matter.

**Temperature 0.7**

*Balanced (default)*

Reasonable variety in phrasing while maintaining accuracy. Good general-purpose setting. Some creative choices in wording.

**Temperature 1.0**

*Creative, variable*

Output varies significantly across runs. May introduce unexpected phrasings or tangents. Higher hallucination risk. Useful for brainstorming, not clinical reporting.

**Takeaway: For clinical applications, use low temperature. Reproducibility = verifiability = safety.**

PHASE 3

# Safety, Governance & Series Bridge

AI safety is not a single topic — it is a design constraint that runs through the entire AI+FHIR series

# The Golden Rule

**Never use LLM output in a clinical context without verifying it against the source data.**

## Validation Strategies:

**Cross-reference against source FHIR data:** Does the summary match the actual resources?

**Structured output validation:** Does the LLM output conform to the expected schema?

**Human spot-checking:** Clinician verifies medications, allergies, active diagnoses

**Reproducibility testing:** Run the same input twice — significant factual differences suggest hallucination

# Hallucination Risks in Clinical Contexts

### Invented Medication

LLM summary says "Metformin 500mg twice daily" — but there is no Metformin MedicationRequest in the FHIR bundle. The LLM inferred it from the diabetes diagnosis.

### Fabricated Lab Value

Summary reports "Most recent A1c: 8.2%" — but no A1c Observation exists. The LLM filled a gap with a plausible-sounding number.

### Patient Conflation

In batch processing, the LLM blends Patient A's medications into Patient B's summary — especially if context windows are reused without clearing.

**Each of these can cause patient harm if a clinician acts on the LLM output without verifying against the source FHIR data.**

# PHI and HIPAA Considerations

Scenario: A developer copies a patient's FHIR Patient/$everything bundle — 300,000 characters of clinical data including names, dates of birth, diagnoses, and medications — and pastes it into ChatGPT to generate a summary.

This is a HIPAA violation. The data is now on external servers, potentially logged, and outside the covered entity's control.

## Local Model (Ollama)

Data stays on your machine

No BAA needed

No risk of external exposure

This is why our exercises use Ollama

## Cloud API (with BAA)

BAA required with API provider

Azure OpenAI offers BAA-backed endpoints

De-identified data may be acceptable

Know where your data goes

**Key principle: Know where your data goes and what protections are in place.**

# Human-in-the-Loop

## AI as Decision Support, Not Decision Maker

**1 LLM Processes**

LLM processes data and generates a summary or suggestion

>

**2 Clinician Reviews**

Clinician reviews output against source FHIR data and clinical context

>

**3 Clinician Decides**

Clinician makes the clinical decision based on verified information

**The Analogy:**

A resident presents a case summary to an attending. The attending doesn't sign off without reviewing the chart. LLM output deserves the same scrutiny.

LLMs cannot assess clinical context not in the data, cannot take responsibility for outcomes, and regulatory frameworks increasingly require human oversight.

# Regulatory Awareness

*Awareness level — know these frameworks exist and why they matter.*

## ONC Health IT Certification (HTI-1)

Transparency requirements for AI-based clinical decision support tools

Disclosure requirements for predictive models in certified health IT

If your AI+FHIR application is part of certified health IT, these rules apply

## FDA — AI/ML as Medical Devices

AI tools for diagnosis/treatment may be regulated as medical devices

FDA has approved hundreds of AI/ML-enabled medical devices

"Patient summary" vs. "clinical recommendation" = different regulatory bar

## When Building AI+FHIR Applications, Ask:

1. Know your intent — informational or clinical?
2. Know your data flow — does PHI leave your infrastructure?
3. Know your audience — will clinicians make decisions from this?
4. Know your obligations — which regulatory frameworks apply?

# Real-World Use Cases: LLMs + FHIR

**Patient Record Summarization**

Mayo Clinic and others use LLMs to summarize records for diagnostic support

**(Our Exercise 3!)**

**Clinical Note Generation**

Ambience, Abridge, Amazon HealthScribe
generate SOAP notes from transcripts

**Natural Language to FHIR Query**

"Find all diabetics with A1c > 8" translated
into FHIR search parameters

**Structured Data Extraction**

LLMs extract medications, conditions, labs
from free-text notes into FHIR resources

**Clinical Trial Matching**

Match patient FHIR profiles to trial eligibility criteria automatically

**Predictive Risk Analysis**

Combine clinical notes + structured data
for early detection (sepsis, readmission)

# Where These Concepts Scale Up

| | | |
|---|---|---|
| **Summarization Pipeline (Exercise 3)** | --> | Becomes a subagent in Part One's multi-agent architecture |
| **Feature Extraction (Exercise 3)** | --> | Input to RAG Lab's embedding and vector store pipeline |
| **Prompt Engineering (Exercise 4)** | --> | Applies to agent prompts in Parts One and Two |
| **Manual FHIR Access (requests library)** | --> | Replaced by MCP — standardized LLM-to-FHIR protocol |
| **Safety Principles (Phase 3)** | --> | Operationalized as least-privilege, validation, audit logging |

**Pattern: Learn the concept → build it manually → see it operate at scale in a production-like architecture.**

# Preview: Model Context Protocol (MCP)

*From manual FHIR calls to standardized LLM-to-server communication.*

### Lesson 17 — Manual Mode

You write the FHIR query code yourself:

You --> Python code --> FHIR Server
You --> extracted data --> LLM
You --> read response --> make decisions

You are the intermediary between
the LLM and the FHIR server.

### Parts One/Two — MCP Mode

The LLM connects to FHIR via MCP:

User --> LLM --> MCP Server --> FHIR
FHIR --> MCP Server --> LLM --> User

MCP server exposes FHIR operations
as tools the LLM can invoke.
The LLM decides which tool to call.

*You need "manual mode" understanding before "autonomous mode" makes sense.*
*When an MCP-connected agent makes an unexpected FHIR query, you need to understand FHIR endpoints and resource types to debug it.*

# Safety as a Series-Long Theme

| | |
|---|---|
| **Hallucination Mitigation** | Part One: structured validation patterns for agent outputs |
| **PHI Protection** | Part One: least-privilege FHIR access scoping for agents |
| **Human-in-the-Loop** | Part Two: human review checkpoints in multi-agent pipelines |
| **Output Validation** | Part One: JSON schema validation of agent tool calls |
| **Audit & Accountability** | Part One: audit logging of all agent actions and FHIR queries |
| **Grounding with Source Data** | RAG Lab: retrieval-augmented generation anchors output to sources |

***Safety in healthcare AI is not about avoiding AI — it's about using it with the same rigor you apply to any clinical tool.***

# Key Takeaways

LLMs are powerful tools for healthcare — AND they require responsible use

Raw FHIR bundles don't fit in context windows — structured extraction solves this

Prompt engineering is a practical skill that directly affects output quality and safety

The "golden rule": never trust LLM output without validating against source data

Local models protect PHI — cloud models require BAAs and careful data handling

Everything you learned today reappears in Parts One, Two, and the RAG Lab

# Thank You

Using LLMs with FHIR

*Next in the series: Agentic AI — Part One*
*Agent architecture, MCP, tool-use, and structured validation*

# Questions & Discussion

Using LLMs with FHIR

*Next in the series: Agentic AI — Part One*
*Agent architecture, MCP, tool-use, and structured validation*